

SQL

Theory Notes

1. TCL – Transaction Control Language

- **Definition:** Commands that manage transactions in a database.
- **Key commands:**
 - `START TRANSACTION` → begins a transaction.
 - `COMMIT` → saves changes permanently.
 - `ROLLBACK` → undoes changes to the last committed state.
 - `SAVEPOINT` → sets a point within a transaction to roll back to.
 - `ROLLBACK TO SAVEPOINT` → undoes changes only up to the savepoint.
- **Use case:** Ensures data consistency and allows safe recovery from errors.

2. DCL – Data Control Language

- **Definition:** Commands that control access to data in the database.
- **Key commands:**
 - `GRANT` → gives privileges to a user.
 - `REVOKE` → removes privileges from a user.
- **Privileges:** `SELECT`, `INSERT`, `UPDATE`, `DELETE`, `ALTER`, `ALL`.
- **Use case:** Security and access control in multi-user environments.

Coding Round Questions

TCL

1. Create a table `accounts` with fields `id` and `balance`. Insert 5 records.
2. Start a transaction and update all balances to 1000. Use `ROLLBACK` to undo changes.
3. Start another transaction, update balance of `id=1` to 50000, set a savepoint, then update balance of `id=2` to 30000. Roll back to the savepoint and commit.

DCL

4. Create two users `user1` and `user2` with passwords.
5. Grant `SELECT` privilege on a table `departments` to `user2`.
6. Grant `ALL` privileges on the database to `user2`.
7. Revoke `SELECT` privilege from `user2`.
8. Drop `user2` from the database.

1. Joins

- **INNER JOIN** → returns rows with matching values in both tables.
- **LEFT JOIN** → returns all rows from left table + matching rows from right.
- **RIGHT JOIN** → returns all rows from right table + matching rows from left.
- **CROSS JOIN** → returns Cartesian product (all combinations).

2. Aggregate Functions

- **COUNT()** → counts rows.
- **SUM()** → adds values.
- **AVG()** → calculates average.
- **MAX()** → highest value.
- **MIN()** → lowest value.

3. GROUP BY and HAVING

- **GROUP BY** → groups rows based on column values.
- **HAVING** → filters groups (like WHERE but for aggregates).

4. Subqueries

- **Definition:** Query inside another query.
- **Types:**
 - Single-row subquery → returns one value.
 - Multi-row subquery → returns multiple values.
 - Correlated subquery → depends on outer query.
- **Operators:**
 - **IN** → matches any value in subquery.
 - **ANY** → compares with any value in subquery.
 - **ALL** → compares with all values in subquery.

Coding Round Questions

1. Write a query to list all students with their department names using **INNER JOIN**.
2. Insert a student without a department and demonstrate **LEFT JOIN**.
3. Show all departments even if no student is assigned using **RIGHT JOIN**.
4. Write a query to count employees in each department and display only those with more than 2 employees.
5. Select employees whose salary is greater than the average salary (use subquery).
6. Find employees in IT whose salary is greater than **ANY** salary in HR.
7. Find employees in IT whose salary is greater than **ALL** salaries in HR.
8. Write a query to display department-wise average salary using a subquery.

Joins.sql

```
CREATE TABLE departments(  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50) NOT NULL,  
    dept_location VARCHAR(50)  
  
)  
  
CREATE TABLE students(  
    stu_id INT PRIMARY KEY ,  
    stu_name VARCHAR(50) NOT NULL,  
    email VARCHAR(100) UNIQUE,  
    salary DECIMAL(10,2) CHECK (salary >0),  
    dept_id INT,  
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)  
  
)  
  
INSERT INTO departments VALUES(1,"HR", "Chennai"),  
(2,"IT","Bengaluru"),  
(3, "Finance", "Delhi");  
  
INSERT INTO students VALUES(101, "Pirithika", "pirithika@gmail.com", 50000,  
2),  
(102, "Suji", "suji@gmail.com", 40000, 1),  
(103, "priya", "priya@gmai.com", 70000, 3),  
(104, "viji", "viji@gmail.com", 30000, 1),  
(105, "maha", "maha@gmail.com", 100000, 2),  
(106, "kaviya", "kaviya@gmai.com", 60000, 3);  
  
SELECT s.stu_name, d.dept_name  
FROM students s  
INNER JOIN departments d ON s.dept_id = d.dept_id;  
  
INSERT INTO students VALUES(107, "Raj", "raj@gmail.com", 50000, NULL)  
  
SELECT s.stu_name, d.dept_name  
FROM students s  
LEFT JOIN departments d ON s.dept_id = d.dept_id;
```

```

SELECT s.stu_name, d.dept_name
FROM students s
RIGHT JOIN departments d ON s.dept_id = d.dept_id;

SELECT s.stu_name, d.dept_name
FROM students s
FULL JOIN departments d ON s.dept_id = d.dept_id;

SELECT s.stu_name, d.dept_name
FROM students s
CROSS JOIN departments d

-- select department which has more than 2 employee
SELECT d.dept_name, COUNT(d.dept_name) AS total_employee
FROM students s
JOIN departments d ON d.dept_id = s.dept_id
GROUP BY d.dept_name
HAVING COUNT(d.dept_name) >2;

INSERT INTO students VALUES(108, "naren", "naren@gmail.com", 80000, 2)

```

Subquery.sql

```

-- subquery

CREATE TABLE employee(
    emp_id INT,
    emp_name VARCHAR(50),
    salary INT,
    department VARCHAR(20)
)

INSERT INTO employee VALUES(1,'Arun', 30000, 'IT'),
(2,"Divya",45000,"HR"),
(3,"Karthik", 50000, "IT"),
(4, "Meena", 35000, "Finance"),
(5, "Rahul", 60000, "IT");

-- DQL

SELECT * FROM employee;

```

```

SELECT MAX(salary) FROM employee;

SELECT AVG(salary) FROM employee;

-- subquery

SELECT emp_name, salary
FROM employee
WHERE salary > (
    SELECT AVG(salary) FROM employee
)

SELECT emp_name, salary
FROM employee
WHERE salary > (
    SELECT AVG(salary) FROM employee
)

SELECT emp_name FROM employee
WHERE department IN ("IT","HR")

SELECT emp_name, salary FROM employee
WHERE salary > ANY (
    SELECT salary
    FROM employee
    WHERE department = "IT"
)

SELECT emp_name, salary FROM employee
WHERE salary > ALL (
    SELECT salary
    FROM employee
    WHERE department = "IT"
)

SELECT emp_name, (SELECT AVG(salary) FROM employee) AS avg_salary
FROM employee;

SELECT department, avg_sal
FROM (
    SELECT department , AVG(salary) AS avg_sal
    FROM employee

```

```
GROUP BY department  
)AS dept_avg;
```

```
-- tcl => TRANSACTION control language  
  
CREATE TABLE accounts(  
    ID INT,  
    balance DOUBLE  
);  
  
INSERT INTO accounts VALUES(1,40000),(2,50000),(3,60000),(4,10000);  
  
SELECT * FROM accounts;  
  
DELETE FROM accounts;  
  
START TRANSACTION;  
  
UPDATE accounts SET balance = 1000;  
  
COMMIT; --SAVES PERMANENTLY  
  
ROLLBACK; -- ROLL BACK TO PREVIOUS STATE  
  
START TRANSACTION;  
UPDATE accounts SET balance = 50000 WHERE id=1;  
  
SAVEPOINT sp1;  
  
UPDATE accounts SET balance = 30000 WHERE id=2;  
  
ROLLBACK TO sp1;  
  
COMMIT;  
  
-- dcl -> data control language--  
  
CREATE user 'pirithika'@'localhost' identified BY '1234';  
  
CREATE user 'maha'@'localhost' identified BY '1234';
```

```
GRANT SELECT ON mysqlpirithika.departments TO 'maha'@'localhost';  
  
GRANT ALTER ON mysqlpirithika.departments TO 'maha'@'localhost';  
  
GRANT UPDATE ON mysqlpirithika.departments TO 'maha'@'localhost';  
  
GRANT DELETE ON mysqlpirithika.departments TO 'maha'@'localhost';  
  
SHOW GRANTS FOR 'maha'@'localhost';  
  
  
GRANT ALL ON mysqlpirithika.* TO 'maha'@'localhost';  
  
  
REVOKE SELECT ON mysqlpirithika.* FROM 'maha'@'localhost';  
  
REVOKE ALL privileges ON mysqlpirithika.* FROM 'maha'@'localhost';  
  
DROP user 'maha'@'localhost';
```